Port Declaration Using Verilog 2001

```
Module module_name
( IN1, IN2, CLK, RST, OUT1, OUT2 );

input   [3:0]   IN1 , IN2 ;
input           CLK ;
output  [7:0]   OUT ;

reg OUT ;

// Code

endmodule
```

Combine
- **Port List**
- **Port Declaration**
- **Port Data Type**

Don't forget semicolon

```
Module module_name (

input   wire   [3:0]   IN1 , IN2 ,
input   wire           CLK ,
output  reg    [7:0]   OUT

);

// Code

endmodule
```

Don't put Comma after last port

© 2022 Eng. Ali Eltemsah

Sensitivity List Using Verilog 2001

```
always @ ( A or B or C or D )
 begin
   E = (A & B) | ( C & D )
 end
```

```
always @ (*)
 begin
   E = (A & B) | ( C & D )
 end
```

- always @ (*) is used to describe combinational logic.
- Simulator will generate the full sensitivity list based on the statements written inside the always block.
- always @ (*) is very useful as it avoid sensitivity list mistakes and bugs.

© 2021 Eng. Ali Mohamed

Blocking Vs Non-blocking Assignment

- There are two types of procedural assignments:-

Blocking assignments:-

- The blocking assignments are indicated by **'='**.
- The blocking statement must be executed before the execution of the statements following it in a sequential block. In other words, it blocks the next statements to be executed until it is executed.
- It is used for to describe multi-level combinational logic.
- Ex: A **=** B & C ;

Non-blocking assignments:-

- The blocking assignments are indicated by **'<='**.
- The Non-blocking statements are executed in parallel within the same procedural block.
- It is used for flip-flop assignments to update all the flops at the same time regardless to the order or dependence upon each other
- Ex: A **<=** B & C ;

© 2022 Eng. Ali Eltemsah

Blocking Vs Non-blocking Assignment

- **Consider the following example:-**

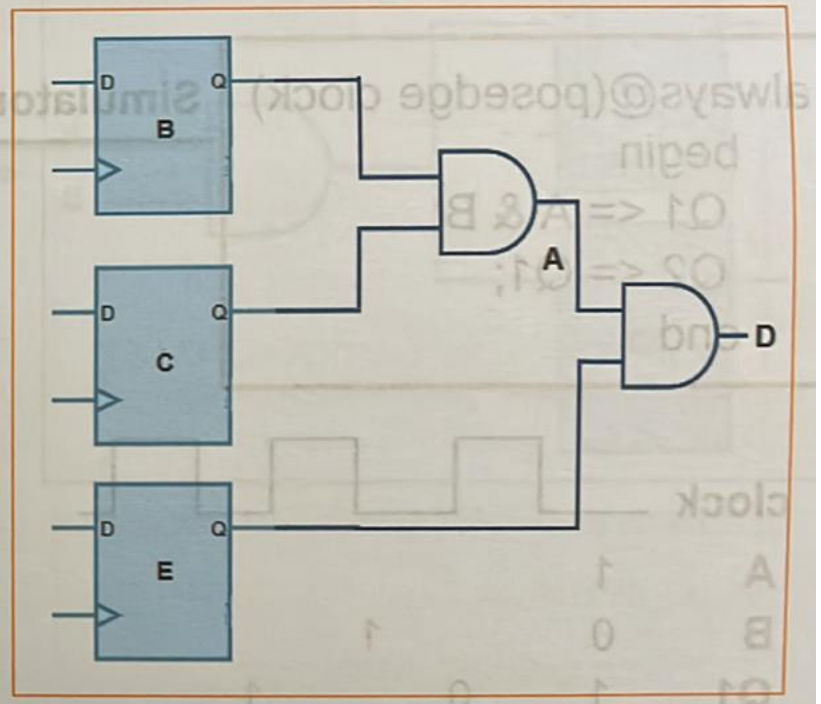
We need to describe the following logic

```
always@(*)
 begin
   A <= B & C;
   D <= A & E;
 end
```

Here D takes the value of A that existed BEFORE this block was triggered, This might be the value of A from the previous clock cycle, This is not work as intended.

```
always@(*)
 begin
   A = B & C;
   D = A & E;
 end
```

Here A changes then D changes, This will work as intended.



Always use blocking assignment "=" to describe combinational logic

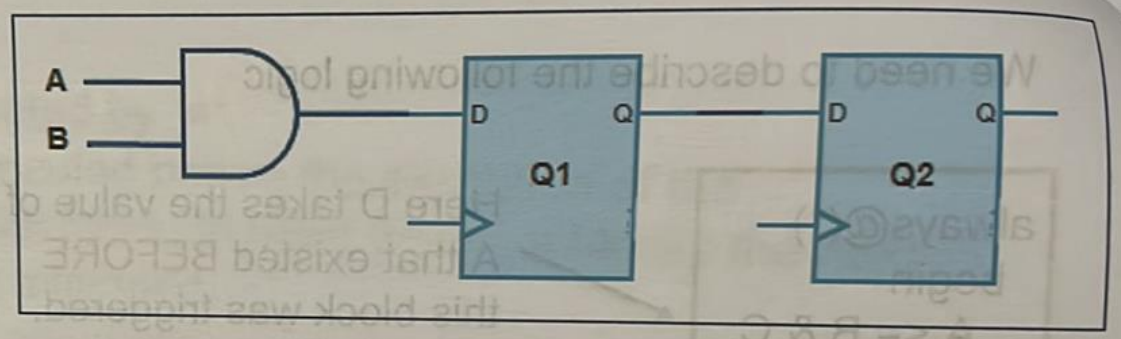
© 2022 Eng. Ali Eltemsah

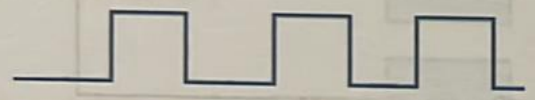
Blocking Vs Non-blocking Assignment

- **Consider the following example:-**

```
always@(posedge clock)
   begin
    Q1 <= A & B;
    Q2 <= Q1;
   end
```

Simulator →





A	1		
B	0	1	
Q1	1	0	1
Q2	1	1	0

Always use Non-blocking assignment "<=" for flip flops assignment

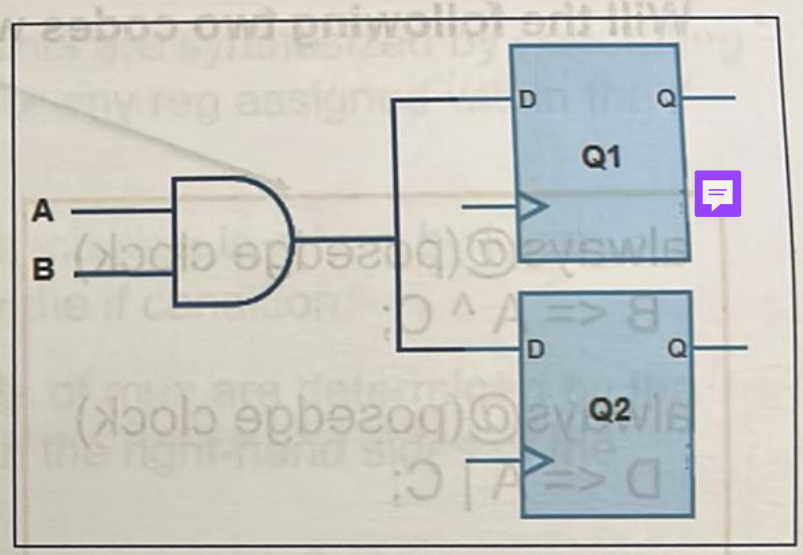
© 2022 Eng. Ali Eltemsah

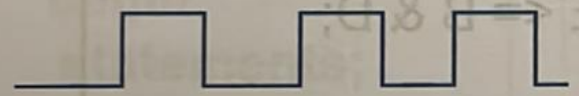
Blocking Vs Non-blocking Assignment

- **Consider the following example:-**

```
always@(posedge clock)
 begin
   Q1 = A & B;
   Q2 = Q1;
 end
```

Simulator →





clock				
A	1			
B	0		1	
Q1	1	0		1
Q2	1	0		1

Always use Non-blocking assignment "<=" for flip flops assignment

© 2022 Eng. Ali Eltemsah

Blocking Vs Non-blocking Assignment

- **Will the following two codes will produce the same logic ?**

```
always@(posedge clock)
  B <= A ^ C;

always@(posedge clock)
  D <= A | C;

always@(posedge clock)
  E <= B & D;
```

```
always@(posedge clock)
  begin
      B <= A ^ C;
      D <= A | C;
      E <= B & D;
  end
```

Yes, as all the Non-blocking assignments will be exceeded in parallel

© 2022 Eng. Ali Eltemsah

Conditional Statements > IF ELSE

➢ **Syntax**

```
if (condition)
  begin
   statements;
  end
else if (condition)
  begin
   statements;
  end
else
  begin
   statements;
  end
```

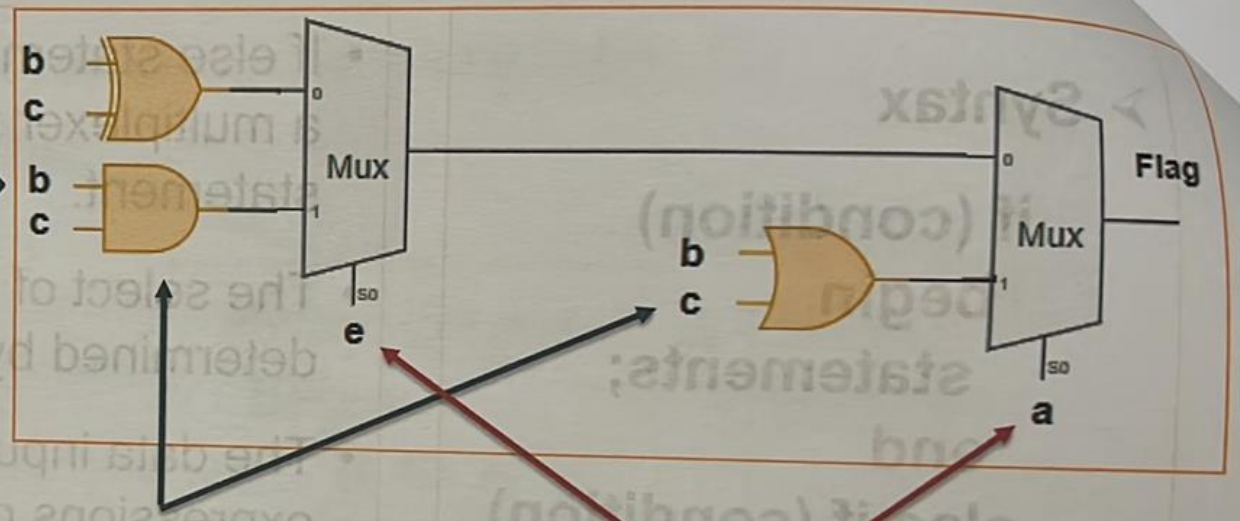
- If else statements are synthesized by generating a multiplexer for any reg assigned within the if statement.
- The select of each mux is driven by logic determined by the if condition.
- The data inputs of mux are determined by the expressions on the right-hand sides of the assignments.
- If else is used only for explicit priorities between different conditions or variables.
- begin … end only needed if more than one statement is under the condition.
- If else statements are **only** used inside always block.

© 2022 Eng. Ali Eltemsah

Conditional Statements > IF ELSE

- **Example**

```
if (a)
  begin
    Flag = b | c;
  end
else if (e)
  begin
    Flag = b & c;
  end
else
  begin
    Flag = b ^ c;
  end
```



Right Hand Side Expressions are inputs of Muxs

Conditions are selection of Muxs

If/elseif/else implement priority multiplexer (high comb depth)

Continuous assignment

- *Verilog* has a short hand way to capture combinational logic structure called **"Continuous assignment"**
- **"assign"** statement is used to implement combinational logic directly.
- **Example:-**

```
reg E ;
always @ (*)
 begin
   E = (A & B) | ( C & D )
 end
```

```
wire E ;
assign E = (A & B) | ( C & D)
```

© 2022 Eng. Ali Eltemsah

Summary

- **RTL Three Coding Styles**

```
always@( ???edge clock)
 begin
  statements ;
 end
```

This block is the only way to build flip-flops as every variable assigned in a block starting with always@(??edge clock) becomes output of an edge-triggered flip-flop,

```
always@(*)
 begin
  statements;
 end
```

This block is used to specify a combinational logic by its behavior

```
assign Y = .....
```

This statement is used to specify a combinational logic by its structure

© 2022 Eng. Ali Eltemsah

Specifying Numbers in Verilog

size ’base value

8’b00001101	10’d5	20	10’b111	16’hF3
Size = 8-bits Base = binary Value = 13	Size = 10-bits Base = decimal Value = 5	Size = 32-bits Base = decimal Value = 20	Size = 10-bits Base = binary Value = 7	Size = 16-bits Base = hexa Value = 243

Notes:-

1. High order bits are truncated if the size is smaller
2. Unspecified high order bits are zero-filled whether the left most bit is 0 or 1

© 2022 Eng. Ali Eltemsah

Testbench

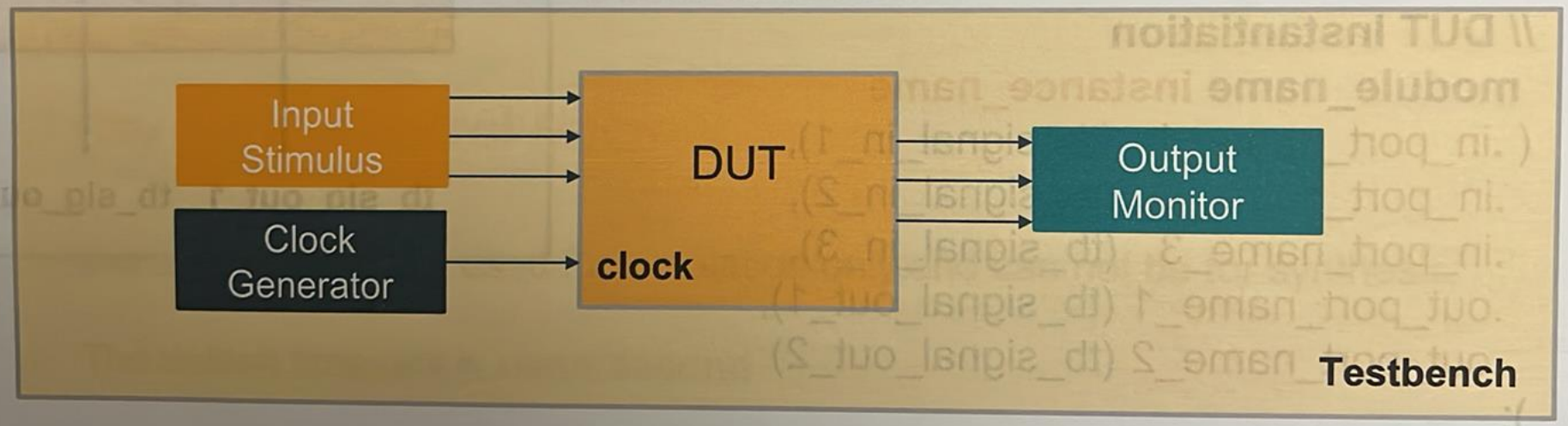
- Testbench is HDL code that allows you to provide a documented, repeatable set of stimuli that is portable across different simulators.
- Testbenches are needed to run simulation on our design to know whether the Verilog code functionally works correctly or not.
- Testbench is very useful to avoid applying input signal stimulus manually especially when the number of input signals are very large.
- Testbenches are used for simulation purpose only **(not for synthesis)**, so many non synthesizable Verilog constructs can be used as (initial block), …, so Testbenches are Not synthesizable.

© 2022 Eng. Ali Eltemsah

Testbench

Testbenches Main Components:-

- Design Under Test (DUT) Instantiation
- Clock Waveform Generator
- Declare Testbench stimulus and monitor signals
- Applying stimulus on Inputs & Monitor the outputs



© 2022 Eng. Ali Eltemsah

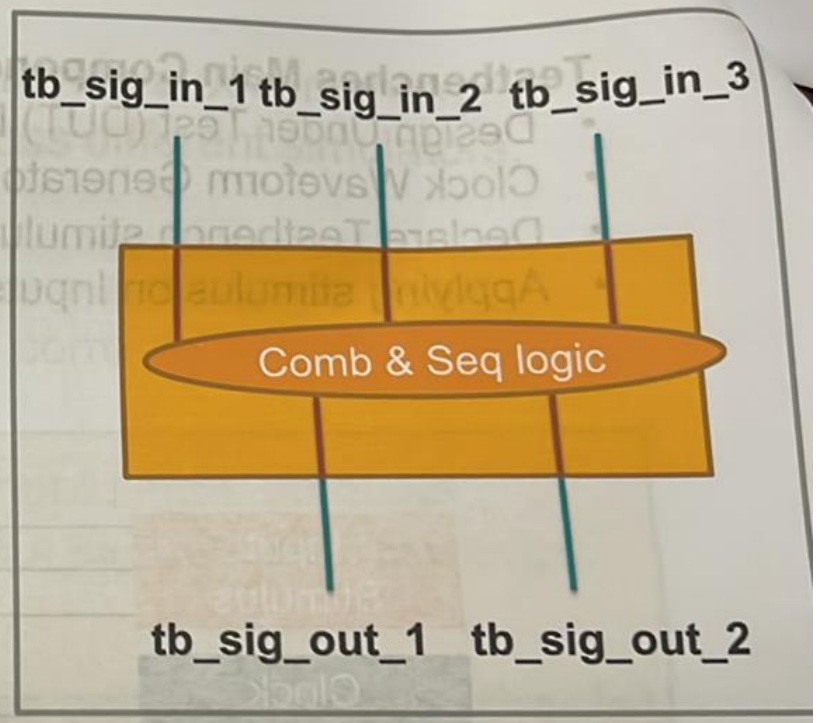
Declare TB Signals & DUT Instantiation

```
// Declare testbench signals
reg  tb_signal_in_1 ;
reg  tb_signal_in_2 ;
reg  tb_signal_in_3 ;
wire tb_signal_out_1;
wire tb_signal_out_2;
```

TB signals connected to DUT input ports must defined as reg

TB signals connected to DUT output ports must defined as wire

```
// DUT Instantiation
module_name instance_name
( .in_port_name_1   (tb_signal_in_1),
  .in_port_name_2   (tb_signal_in_2),
  .in_port_name_3   (tb_signal_in_3),
  .out_port_name_1 (tb_signal_out_1),
  .out_port_name_2 (tb_signal_out_2)
);
```



© 2022 Eng. Ali Eltemsah

System Functions

- **$dumpfile**
  > This function is used to define the name of the vcd dump file
- **$dumpvars**
  > This function is used to dump(save) all the waveforms during the simulation
- **$finish**
  > This function is used to finish and quit the simulation

Delay Time Control (#)

- **#** is a time delay and used for simulation only and can not be for synthesis.
- The default time unit is **nano second**
- **#50** indicates 50 ns delay

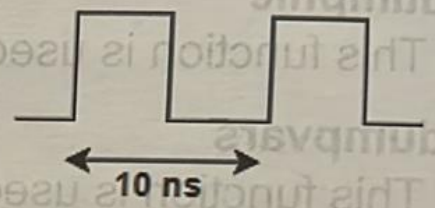
© 2022 Eng. Ali Eltemsah

Clock Generator

```
always  #5    Clock = ~Clock
```

Tperiod = 10 nano sec

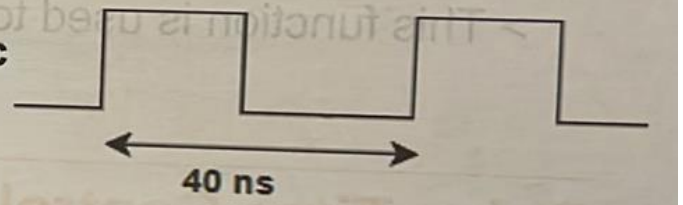
Frequency = 100 M




```
always  #20   Clock = ~Clock
```

Tperiod = 40 nano sec

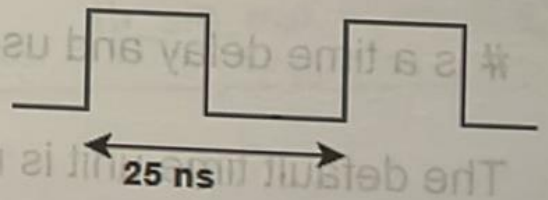
Frequency = 25 M




```
always  #12.5   Clock = ~Clock
```

Tperiod = 25 nano sec

Frequency = 40 M



© 2022 Eng. Ali Eltemsah

Initial block

- Initial block is one of Verilog's procedural blocks (execute statements in sequence)
- Initial block is being used to described the stimulus or test patterns and test scenarios for the design.
- The initial block is executed once and start the simulation at (time zero)

```
initial
 begin
  $dumpfile("count.vcd");
  $dumpvars;
  clock100 = 0;
  latch = 0;
  dec = 0;
  in = 4'b0010;
  #16 latch = 1;
  #10 latch = 0;
  #100 $finish;
 end
```

- Procedural delay (#delay) suspends the execution of the initial block for the given delay time
- All the variables assigned inside the initial block must be of reg Type.
- Not synthesizable

© 2022 Eng. Ali Eltemsah

Tips

Don't assign the same variable in more than one always block.

Don't mix blocking and non-blocking assignments in the same block

Only "reg" type variable can be assigned within "initial" and "always" blocks.

Always block and initial block are called procedural blocks.

© 2022 Eng. Ali Eltemsah